This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

LLM Enabled Resource Allocation for Knowledge Base Federated-Split Learning in Semantic Communications

Kaixiang Yang, Ruikun Li, Yikai Xu

***Abstract*—With the advancement of 6G and IoT, semantic communication has evolved from "bit-focused" transmission to "meaning-centered" delivery, yet faces challenges including limited device resources and privacy risks in multimodal data processing. While semantic knowledge bases enable structured domain guidance for feature extraction, edge computing reduces bandwidth usage through local processing, and federated learning (FL) preserves privacy by aggregating model updates instead of raw data. However, standalone edge computing lacks complex reasoning capability, and FL incurs high communication costs. This paper proposes an LLM-powered resource allocation framework for knowledge base-assisted federated-split learning. First, we design a knowledge-aware federated-split architecture that integrates semantic knowledge bases into model splitting and aggregation. Second, we develop a system model formalizing computation latency, communication rates, and end-to-end latency. Third, we formulate resource allocation as a Constrained Markov Decision Process (CMDP) and adopt a Proximal Policy Optimization (PPO) framework with a frozen LLM to generate knowledge-aware rewards for policy learning. Experiments on the FLASH dataset demonstrate that our framework outperforms baselines by reducing training latency, accelerating convergence, and improving semantic reconstruction accuracy, effectively addressing resource allocation challenges in privacy-preserving semantic communication.**

***Index Terms*—Semantic Communications, Federated-Split Learning, Large Language Model, Resource Allocation, Proximal Policy Optimization.**

I. Introduction

With the advent of 6G and the proliferation of the Internet of Things (IoT), semantic communications have emerged as a pivotal paradigm shift from traditional "bit-centric" transmission to "meaning-centric" data delivery. Unlike conventional communication systems that prioritize error-free bit transfer, semantic communications focus on extracting, transmitting, and reconstructing semantic information—the core meaning embedded in multimodal data. This shift is critical for supporting latency-sensitive and bandwidth-hungry applications, including autonomous driving, remote healthcare, and immersive XR. However, realizing high-performance semantic communications hinges on training robust semantic encoder-decoder models, which face inherent challenges: these models demand substantial computational resources, yet end-user devices are often resource-constrained, and raw multimodal data frequently contains sensitive privacy information that cannot be directly transmitted.

At the heart of effective semantic communications lies the semantic knowledge base—a structured, domain-specific repository that underpins semantic understanding and model training. This knowledge base encapsulates three key components: (1) multimodal feature correlations; (2) semantic concept mappings; and (3) task-aware priors. Che *et al.* [1] proposed a knowledge base deployment mechanism based on edge caching to support the application of semantic communication. Further, for the semantic understanding-oriented requests of intelligent devices, the knowledge base is considered to be deployed in the edge cache in the form of a knowledge graph. Wang *et al.* [2] proposed a codebook-based unified hierarchical semantic knowledge base (UH-SKB) framework for multi-task semantic communications. Without a well-constructed knowledge base, semantic encoder-decoders risk extracting irrelevant features or misinterpreting content, undermining the core value of semantic communications.

Edge computing has become a cornerstone technology for addressing the resource constraints of end-user devices in distributed systems. By deploying lightweight computing capabilities at the "edge" of the network , edge computing enables localized data processing—eliminating the need to offload all computations to remote clouds. Wei *et al.* [3] presented an innovative architecture, called "MVR", contributing to computation offloading in mobile edge computing. Yu *et al.* [4] proposed a semantic caching model in edge computing system that caches domain-specialized general models and user-specific individual models. This approach has the potential to reduce the time and resources required to establish individual KBs while accurately capturing the semantics behind users' messages. However, standalone edge computing struggles with complex tasks like deep semantic reasoning or cross-modal feature fusion, which require significant computational power beyond the capacity of most edge devices.

To address privacy concerns and data heterogeneity in distributed learning, federated learning (FL) has emerged as a privacy-preserving paradigm. Xie *et al.* [5] introduced a federated learning framework to simultaneously detect multiple license plates over different network cameras through semantic communication. Xing *et al.* [6] presented a deep semantic communication system based on federated learning with dynamic model aggregation, namely DeepSC-FedDMA, considering time-varying channel conditions. Simulation results demonstrate that the proposed DeepSC-FedDMA over-

Kaixiang Yang, Ruikun Li and Yikai Xu are with the Key Laboratory of Broadband Wireless Communication and Sensor Network Technology, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (email: 2022010307@njupt.edu.cn).

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

whelms the semantic communication systems with federated averaging and pure local training in terms of mean square error and peak signal-to-noise ratio, under three kinds of channels. However, FL faces a critical limitation: transmitting large model updates incurs high communication overhead, which is prohibitive for resource-constrained semantic communication scenarios where bandwidth is limited. Moreover, FL's parallel training framework often struggles with multimodal data, as it fails to leverage the hierarchical feature processing required for semantic content extraction.

Split learning emerges as a critical distributed learning paradigm tailored to alleviate the resource burden of end devices, making it a natural complement to edge computing in semantic communication scenarios. Wang *et al.* [7] built the first edge-enabled wireless split learning testbed (EWSLT), where the Intel smart edge open platform is integrated with the software-defined radio (SDR)-based 5G OpenAirInterface (OAI) platform, with split learning implemented between 5G-edge server and 5G user equipment (UE). Eldeeb *et al.* [8] presented a TinyML-based semantic communication framework for few-shot wireless image classification that integrates split-learning and meta-learning. They exploited split-learning to limit the computations performed by the end-users while ensuring privacy-preserving. However, standalone split learning faces limitations in multimodal semantic communication systems: it lacks global model consistency across multiple edge devices and struggles to adapt to heterogeneous data distributions across devices—gaps that motivate its integration with federated learning to form the federated-split hybrid architecture.

To overcome the limitations of standalone split learning and federated learning, the federated-split learning hybrid architecture has been proposed—combining the resource efficiency of split learning with the privacy and global consistency of federated learning. Hui *et al.* [9] proposed a distributed federated split learning (DFSL) framework. In terms of framework design, DFSL combines the advantages of SL and DFL, enabling resource-constrained edge devices to participate in the training of complex models while reducing the risks of network bottlenecks and single points of failure during training. Ni *et al.* [10] proposed a federated split learning (FedSL) framework that allows for collaborative healthcare analytics on multiple IoMT devices with limited resources. However, federated-split learning introduces a new critical problem: how to optimally allocate limited resources across nodes to minimize training latency, while balancing trade-offs between computation, storage, communication, and privacy.

In recent years, large language models (LLMs) have revolutionized AI-driven decision-making with their exceptional capabilities in contextual reasoning, scenario understanding, and dynamic evaluation. Unlike traditional rule-based systems or reinforcement learning (RL) with manually designed reward functions, LLMs can interpret complex domain knowledge , assess the feasibility of resource allocation strategies against real-world constraints, and generate adaptive reward signals. This capability eliminates the labor-intensive and error-prone process of hand-crafting rewards, which often leads to "reward hacking". In federated-split semantic learning, LLMs serve as a "smart evaluator": they analyze device-specific states, judge the effectiveness of resource allocation actions, and generate rewards aligned with the core goal of minimizing training latency while respecting privacy and knowledge base guidance. This makes LLMs a powerful tool for solving the resource allocation challenge in federated-split semantic communications.

Against this backdrop, this paper proposes an LLM-enabled resource allocation framework for knowledge base federated-split learning in semantic communications, with the following key contributions.

- Federated-Split Architecture: Vertically splits semantic encoder-decoder into edge/cloud submodels; edge generates "smashed data", cloud uses federated averaging to ensure global model consistency.
- Resource-Privacy System Model: Quantifies computation latency, data rates, and end-to-end latency via math models; clarifies split index's role in balancing computation, storage, communication, and privacy.
- LLM-PPO Resource Allocation: Casts problem as CMDP; uses frozen LLM for dynamic rewards, PPO+GAE for optimal policy, and mitigates LLM positional bias.
- FLASH Dataset Validation: Outperforms baselines in reducing training latency, accelerating convergence, and improving semantic reconstruction accuracy.

The remainder of this paper is structured as follows: Section II presents the system model, including the knowledge-aware federated-split architecture, communication latency models, and optimization problem formulation. Section III details the PPO framework, covering CMDP definition, LLM-based reward generation, and PPO policy updates. Section IV reports simulation results, including performance comparisons under varying resource conditions and baseline comparisons. Finally, Section V concludes this paper.

II. System Model

A. Overview

We consider semantic communication scenarios where consumer devices have limited computation and storage, and the raw signal carries sensitive privacy information. To meet both efficiency and privacy requirements, we adopt a split-federated hybrid architecture as shown in Fig. 1.

The complete semantic encoder-decoder is first vertically split into an edge server and a cloud server. The edge server, placed on the device, performs only lightweight shallow encoding and uploads smashed data instead of raw signals. The cloud sever, placed in the cloud, completes deep encoding, semantic reasoning, and decoding.

1) Unimodal Split Learning: Each modality is trained independently within the knowledge base. For every mini-batch. The edge server forward-propagates up to the cut layer, produces smashed data, and immediately deletes the raw data. The cloud server completes the remaining forward and backward passes and then sends the gradient of the smashed data back. The edge server finishes its backpropagation and updates its local weights. The raw data never leave the device, and only the activation tensors are transmitted.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

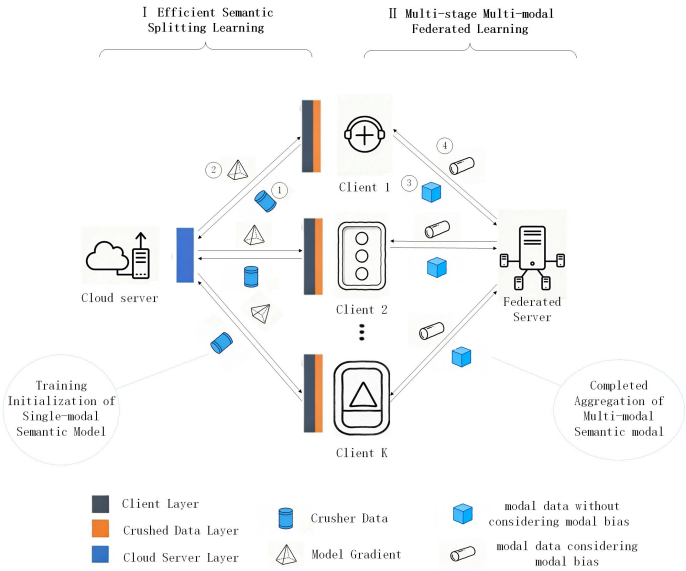

Fig. 1: Federated-Split Hybrid Pipeline for Semantic Codec

2) Multimodal Federated Aggregation: After all unimodal codecs converge, the cloud extracts the semantic representation layers and replicates them across multiple cloud servers. Each device holds a private multimodal dataset. Standard federated averaging is performed on the shared semantic layers while preserving modality-specific edge servers. The updated semantic layers are then distributed back, producing a globally consistent multimodal semantic codec.

3) Model Splitting Strategy: Let the whole deep model be represented by the layer sequence $\{N_0, N_1, \ldots, N_M, \ldots, N_P\}$. We introduce a deterministic cut index M such that Edge subnet: $\{N_0, N_1, \ldots, N_{M-1}\}$; Cut layer: N_M; Cloud subnet: $\{N_{M+1}, N_{M+2}, \ldots, N_P\}$.

The on-device smashed representation is produced by sequential computation through the severing edge followed by the cut layer. The cloud server then takes this broken representation and completes the remaining computation to produce the final output. During split learning, only the broken representation and its gradient traverse the client-cloud link.

B. Resource and Privacy Trade-off

The choice of cut index M directly controls four aspects: On-device compute: determined by the number of layers in the edge server; On-device storage: determined by the parameter count of the edge server; Uplink traffic: proportional to the dimension of the smashed representation; Privacy leakage: shallower cuts retain more private details, deeper cuts discard more. By adjusting M, the framework exposes an explicit trade-off knob between computation, storage, communication, and privacy, which will be optimized as follows.

Semantic communication systems are usually deployed on resource-constrained consumer devices. To keep the client lightweight, we vertically split the semantic encoder-decoder into an edge submodel and a cloud submodel. The edge server only executes the first few layers, while all heavy computations are off-loaded to the cloud. This strategy reduces both FLOPs and the memory footprint on the device.

Transmitting raw sensory data can leak sensitive privacy semantics. With the proposed split, the client uploads only intermediate activations produced by the cut layer, never exposing the original data. Consequently, the attack surface narrows and the probability of privacy leakage is lowered.

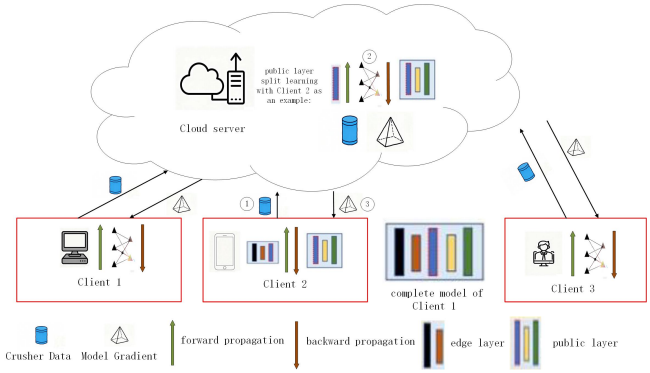

Fig. 2: Architecture of efficient semantic split learning.

1) Semantic Split Learning: An overview of the training flow is given in Fig 2. The procedure is divided into four synchronized stages:

1) **Stage 1: Client forward:** the edge sub-model $\{N_0, N_1, \ldots, N_M\}$ processes the local sample and produces the *shredded* activation $\mathbf{A}$.
2) **Stage 2: Cloud forward:** the cloud sub-model $\{N_{M+1}, \ldots, N_P\}$ takes $\mathbf{A}$ and continues forward propagation to obtain the prediction $\widehat{Y}$.
3) **Stage 3: Cloud backward:** the loss $l(\widehat{Y}, Y)$ is computed and gradients are propagated back to the cut layer, yielding the gradient $\nabla_{\mathbf{A}}$.
4) **Stage 4: Client backward:** the edge sub-model receives $\nabla_{\mathbf{A}}$ and finishes its own backpropagation, updating its weights locally.

Forward propagation of the whole network can be written as

$$F(X) \leftarrow N_P\big(N_{P-1}\big(\ldots N_M(X)\ldots\big)\big). \tag{1}$$

Likewise, the back-propagation chain is executed layer-wise in reverse order:

$$\nabla F^T \leftarrow N_M^T\big(N_{M+1}^T\big(\ldots N_P^T\big)\big). \tag{2}$$

Weight update. Let $H_{u,i}^{r,e}$ denote the edge sub-model of client i at round r and epoch e. After receiving the gradient of the shredded data, the client performs

$$H_{u,i}^{r,e+1} = B\big(H_{u,i}^{r,e}, \zeta_r, \nabla Q_{N_i}^{r,e}\big), \tag{3}$$

where $B(\cdot)$ stands for the local optimizer, ζ_r is the learning rate, and $\nabla Q_{N_i}^{r,e} = Q_{N_i}^{r,e} - \widehat{Q}_{N_i}^{r,e}$ is the activation difference computed by the server.

Symmetrically, the cloud maintains its own submodel $H_{s,i}^{r,e}$. The server-side weights are updated with

$$H_{s,i}^{r,e+1} = B\big(H_{s,i}^{r,e}, \zeta_r, \nabla l(H_{u,i}^{r,e}, \widehat{Y}, Y)\big), \tag{4}$$

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

where the loss $l(\cdot)$ is evaluated on the current prediction $\widehat{Y}$ and the ground-truth semantic label Y shared by the client.

Since raw data never leave the device and only shredded activations are exchanged, the above protocol achieves privacy-preserving, communication-efficient training for semantic communication.

C. Communication Model

1) Computation Latency at the Edge: We model the time required by user i to finish the first h_i layers as a shifted exponential random variable:

$$P\big[\mu_{n_i}^{\mathrm{cp}} < \theta\big] = \begin{cases} 1 - e^{-\frac{\rho_i}{d_{n_i}}(\theta - c_i d_{n_i})}, & \theta \geq c_i d_{n_i} \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

In (5), $c_i d_{n_i}$ is the minimum latency dictated by the chip capability and workload; ρ_i captures the uncertainty caused by dynamic background tasks. And d_{n_i} is the deterministic FLOP count of the first n_i layers, calculated by

$$d_{n_i} = \sum_{p=1}^{n_i} \sum_{q=1}^{n_p} r_p^q \, s_p^q \, n_{p-1} \, l_p^q \, w_p^q \, m, \tag{6}$$

where $r_p^q \times s_p^q$ is the kernel size, n_p the channel number, $l_p^q \times w_p^q$ the output feature map, and m the mini-batch size.

The intermediate tensor resulting that must be uploaded is

$$V_{n_i} = l_{n_i} \, w_{n_i} \, n_{n_i} \, m. \tag{7}$$

2) Uplink/Downlink Data Rate: We adopt OFDMA and assign each device an orthogonal band B_i such that

$$\sum\nolimits_i B_i \leq B. \tag{8}$$

The achievable rates are

$$r_i^u = B_i \log_2\Big(1 + \frac{P_i^u \alpha_i^u}{B_i N_0}\Big), \tag{9}$$

$$r_i^d = B_i \log_2\Big(1 + \frac{P_i^d \alpha_i^d}{B_i N_0}\Big), \tag{10}$$

with P_i^u, P_i^d, α_i^u, α_i^d and N_0 being fixed during one round.

3) End-to-End Delay per Round: Let T_r be the duration of round r. The total training time is

$$T = \sum_{r=1}^{R} T_r = \sum_{r=1}^{R} \max_i T_{i,r}. \tag{11}$$

Because rounds are independent, we minimize $\max_i T_{i,r}$ in each round and drop the index r for brevity.

For client i the round latency is

$$T_i = T_i^u + T_i^d + \sum_{e=1}^{\varepsilon_i} T_i^e + T^a, \tag{12}$$

where $T_i^u = V_{n_i}/r_i^u$ is upload smashed data, $T_i^d = V_{n_i}/r_i^d$ is download gradient, T^a is the aggregation of the server for all devices, T_i^e is a delay of one epoch.

Each epoch contains four sequential steps: 1) forward / backward client on the first n_i layers, 2) uplink transmission of activations, 3) forward / backward server on the remaining layers, and 4) downlink transmission of gradients. Hence,

$$\begin{aligned} T_i^e &= \frac{h_i^{e,d}}{d_i} + \frac{h_i^{e,r}}{r_i^u} + \frac{h_i^{e,D}}{D_i} + \frac{h_i^{e,R}}{r_i^d} \\ &= \frac{p_i \sum_{l=1}^{n_i} V_d^l}{d_i} + \frac{p_i \sum_{l=1}^{n_i} V_r^l}{r_i^u} \\ &\quad + \frac{p_i \big(V - \sum_{l=1}^{n_i} V_d^l\big)}{D_i} + \frac{p_i \sum_{l=1}^{n_i} V_r^l}{r_i^d}, \end{aligned} \tag{13}$$

where d_i means local CPU capability (FLOP,s^{-1}), D_i is the cloud CPU quota assigned to user i, V_d^l, V_r^l are FLOPs and bytes of layer l, p_i is the number of local epochs (fixed for the round).

Because data sets and hardware are heterogeneous, a uniform split layer or bandwidth allocation lengthens the slowest client and hurts accuracy. We therefore treat n_i and R_i as optimization variables and jointly minimize (11) under constraints (8) and $\sum_n x_i^n = 1$, $x_i^n \in \{0,1\}$, where $x_i^n = 1$ iff layer n is selected as the cut for client i.

D. Problem Formulation

The ultimate objective of semantic split learning is to minimize total training time. Given that each round's training duration is independent, this global optimization reduces to minimizing the per-round total time—encompassing both computational processing and communication overhead—formulated as follows:

$$\min T = \min \sum_r \max_i T \tag{14}$$

To linearize the optimization problem, this paper introduces an auxiliary variable T_{max} that dominates the maximum training time among all clients per round. Consequently, the optimization problem can be reformulated as follows.

$$\begin{aligned} & \min_{x_i^l C_i} T_{max} \\ s.t. \quad & C1: T_i \leq T_{max}, i \in \{1, ..., S\} \\ & C2: \sum_i D_i \leq D_{total} \\ & C3: d_i \leq d_{max} \\ & C4: \sum_{n=1}^{N} x_i V_i^n \leq g_i \\ & C5: \sum_{n=1}^{N} x_i^n = 1, x_i^n \in \{0,1\} \\ & C6: \sum_i B_i \leq B. \end{aligned} \tag{15}$$

Let D_{total} denote the total computational resources available on the server, V_i represent the data size of the client-side semantic encoder-decoder model for user i, and g_i, indicate the storage space available on the user terminal i. In this paper, $x_i = \{x_i^1, ..., x_i^N\}$ represents the layer splitting indicator vector, where x_i^n indicates whether the semantic encoder-decoder model is split in layer h: $x_i^n = 1$ indicates that a split

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

occurs in layer n, while $x_i^l = 0$ does not indicate that there is no splitting in that layer. C1 imposes an upper bound on the total training latency per round. C2 ensures that the sum of computational resources allocated to all clients does not exceed the server's total available capacity. C3 requires that during model training on each client device, the computational resources utilized must remain below its local processing capability ceiling. C4 requires that the memory footprint of the client models do not exceed the storage capacity on the device. C5 restricts the semantic encoder-decoder architecture to have exactly one splitting layer. C6 requires that the aggregate bandwidth of all sublinks does not exceed the total available link capacity.

III. LLM-Assisted PPO for Federated-Split Learning

A. MDP on Split Learning in Semantic Communications

The semantic knowledge base constructed in Section II serves as the training corpus for the first-stage encoder–decoder tuning. To decide jointly where to divide and how many resources to allocate, we cast the optimization per round as a restricted Markov decision process (MDP) and solve it with a proximal policy optimization agent (PPO).

1) MDP Formulation: The underlying Markov Decision Process is defined by the tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle. \tag{16}$$

1) State space $\mathcal{G}$ (continuous):

$$g_t = \{P_{i,t}, T_{i,t}\}_{i=1}^{N}, \tag{17}$$

where $P_{i,t}$ is a semantic tag with a short textual hash that describes the personalized model on client i at round t and $T_{i,t}$ is the wall clock latency experienced by client i in round t.

2) Action space $\mathcal{U}$ (continuous):

$$u_t = \{d_{i,t}, D_{i,t}, x_{i,t}V_{i,t}^n, B_{i,t}\}_{i=1}^{N}. \tag{18}$$

where $d_{i,t}$ are local CPU cycles devoted to the first n_i layers. $D_{i,t}$ is cloud CPU cycles assigned to the remaining layers. $x_{i,t}V_{i,t}^n$: on-device memory footprint ($x_{i,t} \in [0,1]$). $B_{i,t}$ is OFDMA bandwidth allocated to client i.

3) Transition kernel $\mathcal{P}$:

$$\mathcal{P}(g_{t+1}|g_t, a_t). \tag{19}$$

where is unknown but sampleable. The executing u_t yields the next state g_{t+1} through the four-stage split learning pipeline.

4) Reward $\mathcal{R}$: Instead of hand-crafting a reward that may suffer from reward hacking, we let a frozen large language model (LLM) score the immediate quality of the action. The prompt sent to the LLM is

$$\texttt{Prompt} = \Big[P_{i,t},\ T_{i,t},\ \pi_\theta(u_t|g_t)\Big], \tag{20}$$

and the returned scalar s_t is mapped to a reward

$$r_t = \texttt{Rule}(s_t) \in [0,1], \tag{21}$$

where $\texttt{Rule}(\cdot)$ is a monotonic normalizer, detailed in Algorithm 1. After K episodes, we obtain the reward sequence.

$$\mathcal{R} = \{r_1, r_2, \ldots, r_K\}, \tag{22}$$

which is discounted by factor γ for policy-gradient computation.

2) Policy Learning via PPO: The agent parameterises a stochastic policy $\pi_\theta(u|g)$ with a deep neural network. PPO is used to maximize the clipped surrogate objective.

$$\mathcal{L}^{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{23}$$

where $\rho_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and $\hat{A}_t$ is the generalized advantage estimator.

MDP constraints are enforced via Lagrangian relaxation; the Lagrange multipliers are updated in the outer loop. After convergence, the policy outputs the optimal continuous action for each client in every communication round, thereby minimizing the global training time while keeping model fidelity intact.

B. Overall Workflow of the Proposed Method

Fig. 3 summarises the full loop that decides server CPU $D_{i,t}$, client memory $x_{i,t}V_{i,t}^n$ and bandwidth $B_{i,t}$ for every round of semantic split learning.

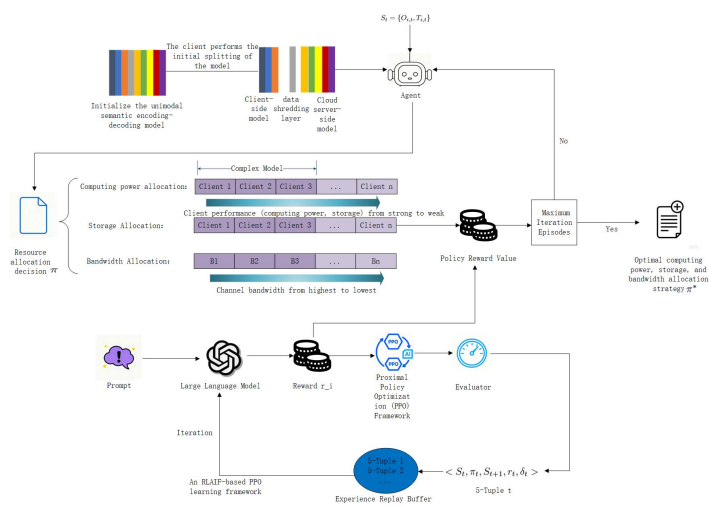

Fig. 3: PPO framework for joint allocation of compute, storage and bandwidth.

1) State observation: Before round t starts, the agent receives

$$g_t = \{P_{i,t}, T_{i,t}\}_{i=1}^{N}, \tag{24}$$

where $P_{i,t}$ is the semantic model tag and $T_{i,t}$ the latency of the previous round.

2) PPO decision: The policy network π_θ outputs the continuous action.

$$u_t = \{D_{i,t}, x_{i,t}V_{i,t}^n, B_{i,t}\}_{i=1}^{N}, \tag{25}$$

that satisfies the requirements of the test, so that

$$\sum\nolimits_{n=1}^{N} x_{i,t}^n = 1, x_{i,t}^n \in \{0,1\}, d_{i,t} \le d_{\max}, \sum\nolimits_i B_{i,t} \le B. \tag{26}$$

3) Environment execution: The four-stage split learning pipeline is executed with the allocated resources. The wall clock latency $T_{i,t}$ of every client is recorded.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

4) LLM reward generation: A frozen large language model is prompted with the fixed template,

$$\texttt{prompt}_{i,t} = \Big\{P_{i,t}, \pi_t, T_{i,t}\Big\}. \tag{27}$$

Using its chain-of-thought reasoning, the LLM returns a scalar z_t that is normalized to the reward,

$$r_t = \texttt{Rule}(s_t) \in [0,1]. \tag{28}$$

No hand-made reward function is used; the signal is *dynamically* generated from the textual description.

5) Experience storage: The evaluator computes an additional score δ_t (advantance estimate) and stores the tuple,

$$(g_t, u_t, g_{t+1}, r_t, \sigma_t), \tag{29}$$

in the replay buffer.

6) Policy update: PPO is executed on the buffer to maximize the clipped surrogate objective while respecting the CMDP constraints via Lagrangian relaxation. The updated policy is implemented in the next round.

After convergence, the agent produces the optimal stationary policy,

$$\pi^* = \Big(D_i, \sum\nolimits_{n=1}^{N} x_i V_i^n, B_i\Big), \tag{30}$$

which minimizes the expected per-round latency while guaranteeing memory, compute, and bandwidth limits.

C. Proximal Policy Optimization (PPO) Algorithm Workflow

The PPO agent maximizes the expected advantage while respecting three hard constraints: cloud–edge compute budget: $\sum_i D_i \le D_{\text{total}}$ and $d_i \le d_{\max}$; edge storage cap: $\sum_{n=1}^{N} x_i V_i^n \le g_i$; bandwidth limit: $\sum_i B_i \le B$.

Let $\pi_{\theta_{\text{old}}}$ denote the policy that collected the data, and π_θ the policy to be optimized. The importance-sampling ratio is written as

$$\varphi_t(\theta) = \frac{\pi_\theta(u_t|g_t)}{\pi_{\theta_{\text{old}}}(u_t|g_t)} = \frac{\pi_\theta\big(D_i, \sum_{n=1}^{N} x_i V_i^n, B_i\big)}{\pi_{\theta_{\text{old}}}\big(D_i, \sum_{n=1}^{N} x_i V_i^n, B_i\big)}. \tag{31}$$

We approximate the advantage with the generalized advantage estimator (GAE),

$$Y_t^{\alpha,\lambda} = \sum_{k=0}^{T-t-1} (\alpha\lambda)^k \sigma_{t+k}, \sigma_t = r_t + \alpha V(g_{t+1}) - V(g_t), \tag{32}$$

where $\lambda \in [0,1]$ trades bias for variance.

The clipped surrogate objective is

$$L^{\text{CPI}}(\theta) = \mathbb{E}_t\Big[\min\Big(\varphi_t(\theta) Y_t^{\alpha,\lambda}, \text{clip}\big(\varphi_t(\theta), 1-\epsilon, 1+\epsilon\big) Y_t^{\alpha,\lambda}\Big)\Big]. \tag{33}$$

A critic network $V_\phi(g)$ is trained to regress the discounted return

$$S_t = \sum_{k=0}^{T-t-1} \alpha^k r_{t+k+1}, \tag{34}$$

via the mean-squared-error loss

$$L^{\text{VF}}(\phi) = \mathbb{E}_t\Big[\big(V_\phi(g_t) - S_t\big)^2\Big]. \tag{35}$$

The immediate reward r_t is produced by the LLM prompt $\{P_{i,t}, \pi_t, T_{i,t}\}$. Therefore, L^{VF} is directly related to the signal generated by the LLM.

The total loss aggregates policy improvement, value fitting, and entropy regularization:

$$L^{\text{Total}}(\theta,\phi) = -L^{\text{CPI}}(\theta) + d_1 L^{\text{VF}}(\phi) - d_2 \mathbb{E}_t\big[\mathcal{H}\big(\pi_\theta(\cdot|g_t)\big)\big], \tag{36}$$

with $d_1, d_2 > 0$ balancing the three terms.

D. LLM-Based Reward Generation for PPO

Rather than hand-engineering a reward function, we let a frozen LLM emit the per-round reward signal r_t used by PPO. An evaluator continuously assesses the LLM input-output behavior and stores the quintuple,

$$(g_t, u_t, g_{t+1}, r_t, \sigma_t), \tag{37}$$

in the replay buffer. Prompt engineering is refined iteratively from these logs.

We first condense the communication scenario into a single paragraph that is accompanied by every prompt:

We perform cloud-edge split learning of a semantic encoder–decoder among K clients (indexed by $i = 1, \ldots, K$) and one cloud server. Each client owns a private dataset and heterogeneous compute, memory, and bandwidth budgets. The agent decides the split layer n_i, the local CPU cycles d_i, the cloud CPU quota D_i, and the OFDMA bandwidth slice B_i for each client. The goal is to minimize the maximum per-round latency while respecting hard resource caps.

The instance-specific prompt sent to the LLM is then discarded,

$$\texttt{prompt}_{i,t} = \Big\{\underbrace{P_{i,t}}_{\text{model tag}}, \underbrace{\pi_t}_{\text{proposed action}}, \underbrace{T_{i,t}}_{\text{observed latency}}\Big\}. \tag{38}$$

Required LLM reasoning chain: We parse the semantic tag $P_{i,t}$ and infer the real-world use case. In addition, we judge whether the proposed policy π_t is feasible for the inferred scenario. Then, we compare the observed latency $T_{i,t}$ with the empirical requirement of that use case. Next, we give a scalar reward $r_t \in [0,1]$ together with a one-sentence justification.

E. Positional-Bias Mitigation and Evaluation

In practice, we observe that the LLM tends to overweight the *first* element of the prompt triplet. To cancel this positional bias, we run the model twice and average the results:

$$r_t = \frac{1}{2}\Big(r\big[P_{i,t}, \pi_t, T_{i,t}\big] + r\big[T_{i,t}, \pi_t, P_{i,t}\big]\Big). \tag{39}$$

The LLM outputs a single integer $z_t \in [1, 10]$; its weighted score is computed from the token-level probability distribution:

$$r_t(y|x) = \sum_{s=1}^{10} s\, P(s|y,x). \tag{40}$$

The scalar r_t is fed directly into the PPO update for the current resource allocation policy $\pi_t = (D_{i,t}, \sum_{n=1}^{N} x_{i,t} V_{i,t}^n, B_{i,t})$.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

Algorithm 1: PPO for Semantic Split Learning

Input: $d_{n_i}, V_{n_i}, P_{i,t}$
Output: Optimal $\pi^* = (D_i, x_i^n, B_i)$
Initialise: random split n_i, bandwidth B_i;
for *round* $t = 1, 2, \ldots$ **do**
  $g_t \leftarrow \{P_{i,t}, T_{i,t}\}$;
  $u_t \leftarrow \pi_\theta(\cdot|g_t)$; // resources D_i, x_i^n, B_i
  Run split learning; record $T_{i,t+1}$;
  $r_t \leftarrow \frac{1}{2}\big[\mathrm{LLM}(P, \pi, T) + \mathrm{LLM}(T, \pi, P)\big]$;
  $\sigma_t \leftarrow$ align_eval(P, T, π, r_t);
  Store $(g_t, u_t, g_{t+1}, r_t, \sigma_t)$;
  if $\sigma_t < 1$ **then**
    refine LLM prompt;
  Update θ via PPO with GAE;
  Enforce constraints:;
    $\sum_i D_i \le D_{\text{total}},\ d_i \le d_{\max},\ \sum_n x_i^n V_i^n \le g_i,\ \sum_i B_i \le B$;
return π^*;

The evaluator receives the tuple, such as

$$\texttt{value}_t = \big\{P_i, T_{i,t}, \pi_t, T_{i,t+1}, r_t\big\}, \tag{41}$$

and measures how well the LLM reward agrees with human preferences using the AI alignment metric.

Human preferences are converted into hot vectors. The LLM probability vector $P^{\text{AI}} \in \mathbb{R}^{S\times 2}$ is also binarized by taking the arg-max. The alignment score is

$$\sigma = \frac{1}{G}\sum_{x=1}^{G} \mathbb{1}\left[\arg\max_y P^{\text{AI}}_{x,y} = P^{\text{N}}_x\right], \tag{42}$$

where G is the number of comparisons and $P^{\text{H}} \in \{0,1\}^G$ is the human reference.

After every round, we store the quintuplet,

$$\beta_t = (G_t, \pi_t, G_{t+1}, r_t, \sigma_t), \tag{43}$$

in the replay buffer. High σ_t indicates reliable LLM outputs. Low σ_t triggers prompt-engineering updates for subsequent episodes. The buffer serves simultaneously as the living preference data set for the current semantic model.

IV. Simulation Results

Since the semantic communication system involves intermodal signal conversion, its encoding/decoding models are trained on multimodal data. In this paper, we conduct simulation experiments using the widely adopted FLASH dataset to evaluate our proposed semantic disentanglement learning scheme within the semantic communication framework. This dataset comprises GPS, radar, and camera data collected at a frequency of 10Hz. Each sample consists of a 64-dimensional ground truth vector along with synchronized multimodal sensor inputs: GPS data shaped as (1,2), radar point clouds structured in (20,20,20), and RGB images formatted at (3,360,640). In this chapter, we distribute scene-specific data across client nodes by partitioning the entire dataset into 20 nodes. The allocation ratio among nodes containing multimodal data (4 parts), GPS-only data (2 parts), radar-only data (2 parts), and image-only data (2 parts) follows a proportional split of 4:2:2:2, yielding a total of 3,290 samples. This chapter employs CNN layers for deep feature extraction and RNN layers to capture temporal dynamics, supplemented by two fully connected layers serving as classifiers. Specifically, for unimodal feature encoders: 2D-CNN is applied to GPS data, while 3D-CNN processes both LiDAR point clouds and imagery data. The reward model is the open source large language model. The average reward value is defined similarly to $\frac{\sum_{x=1}^{X} reward(x)}{X}$, where x denotes the current training episode and X represents the total number of training episodes. In this

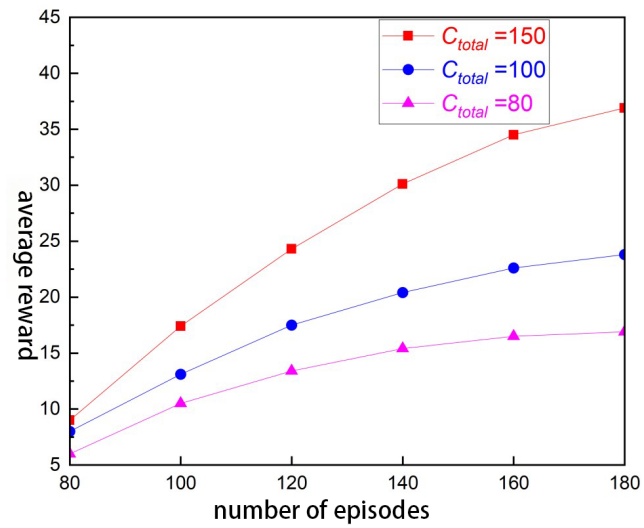

Fig. 4: Plot of average reward versus training episodes under different computational resources allocated by the cloud server

paper, the power spectral density of additive Gaussian noise is set to $N_0 = -144$dBm. Set the maximum client computing capability as $d_{max} = 1.5$ MFLOPS. Set the client computing fluctuation value as $c_i = 0.05$ MFLOPS. In this paper, the transmit power of the uplink is set at $P_i^u = 27$ dBm, while the transmit power of the downlink is set to $P_i^d = 29$dBm. Both uplink and downlink channels share identical path loss characteristics with a link budget gain of $\alpha_i^u = \alpha_i^d = -120$dB.

As illustrated in Figure 4, the plot shows the average reward versus the training episodes under varying total server computational resources. Specifically, we configured three scenarios with server capacities $D_{total} = 80, 100, 150$ MFLOPS, while maintaining fixed client-side parameters: total storage capacity $g = 200$GB and aggregate bandwidth $B = 20$Hz. It can be observed that higher total computational resources allocated by the server to clients result in increased average reward values, yet exhibit no significant impact on the convergence speed of the reinforcement learning (RL) model. This shows that increasing the total computational resources allocated by the server to clients leads to superior performance of the semantic disentanglement learning-based communication resource allocation scheme. This is attributed to the fact that

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

the cloud server's computational resources enable offloading more model training tasks from clients during the Semantic Learning (SL) process.

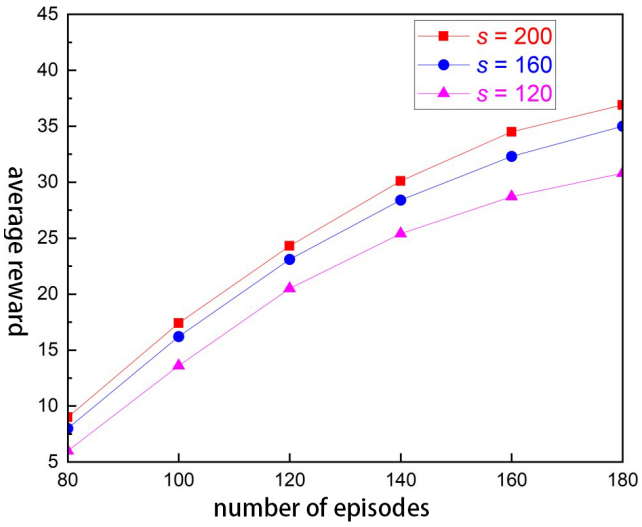

Fig. 5: Plot of average reward versus training episodes under different client storage capacities

As shown in Fig.5, it illustrates the curves of average reward versus episode count under different client storage capacities. The parameters are set as follows: $g = 120, 160, 200$GB; total server computing resources $D_{total} = 150$ MFLOPS; and total bandwidth $B = 20$Hz. It can be observed that the average reward increases as the client's storage capacity grows larger. Observing the curves, it can be seen that the red curve exhibits a convergence trend around 140 episodes, while both the blue and purple curves show convergence tendencies approximately at 120 episodes. From this we can conclude that larger client storage capacities lead to better model performance; conversely, when client storage capacity is limited, the convergence speed of the RL model can be accelerated. This is because a larger client storage capacity expands the exploration space of agents for diverse strategies; whereas smaller client storage capacities facilitate quicker identification of locally optimal strategies by agents.

As shown in Fig.6, it presents the curves of average reward versus episode count under different bandwidth conditions. The total computing resources of the server are set to $D_{total} = 150$ MFLOPS, while the total storage capacity of the clients is configured as $g = 200$GB, with the overall bandwidth set at $B = 20, 15$ and 10Hz, respectively. It can be observed that while the average reward slightly increases with greater channel bandwidth, all scenarios exhibit convergence trends around 120 episodes. From this we can conclude that under practical channel conditions, the bandwidth has a minimal impact on the semantic splitting learning architecture. This is attributed to the fact that during training processes, channel latency constitutes only a minor proportion of total training latency, whereas computational latency accounts for the predominant share.

As shown in Fig.7, it presents the curves of the average reward versus the number of episodes under different evaluation metrics. Set the total computational resources of the server as $D_{total} = 150$ MFLOPS, the total storage capacity of the clients as $g = 200$ GB, and the total bandwidth as $B = 20$Hz.The red curve represents the average reward generated under the AI label alignment method proposed in this chapter. The evaluator can also be other large models or AI agents. In this paper, comparative experiments are conducted using GPT-4o (developed by OpenAI) and Gemini 1.5 Pro (developed by Google DeepMind) as distinct evaluators. Before formal training, large models used in comparative experiments undergo pre-training following an approach similar to that of reward models: defining the communication environment, structuring input prompts and setting output formats. The objective of this comparative experiment is to verify whether employing fixed binary rules as evaluators or utilizing other LLMs for reward model evaluation yields superior perfor-

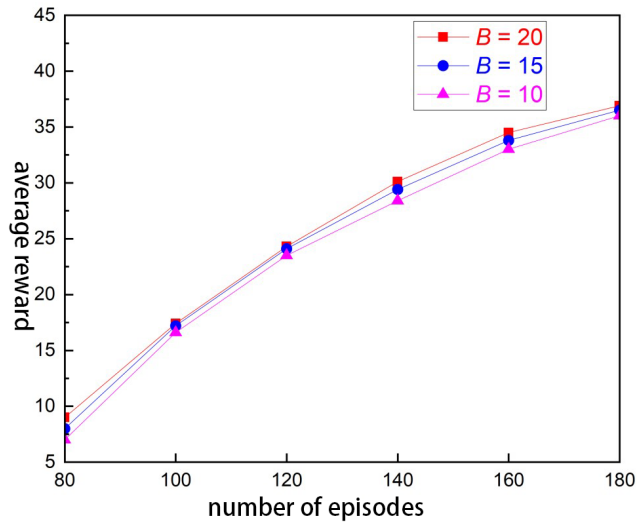

Fig. 6: Average reward per episode versus number of episodes under different bandwidth conditions

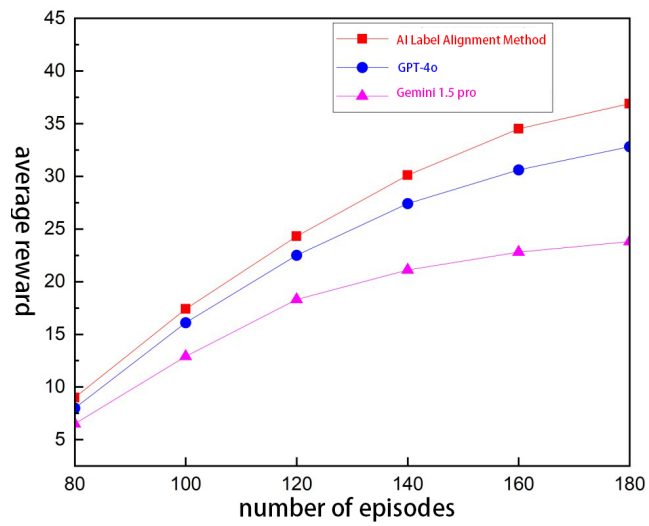

Fig. 7: Average reward per episode versus number of episodes under different evaluation metrics

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

mance within the semantic segmentation learning architecture proposed in this chapter. It can be observed that, under the evaluation metrics proposed in this chapter, the average reward versus episode curve does not exhibit significant global convergence within 180 episodes. In contrast, all baseline evaluation rules show clear convergence trends within the same period, with their convergence values consistently lower than those achieved by the proposed method. This leads to the conclusion that within the semantic segmentation learning architecture proposed in this chapter, the evaluation rules are better suited for the AI label alignment method. Potential reasons include: (i) large models such as GPT-4o and Gemini 1.5 Pro exhibit pronounced ”hallucination” tendencies; (ii) the selected evaluator models demonstrate relatively low accuracy in comprehending the optimized objectives defined in this study.

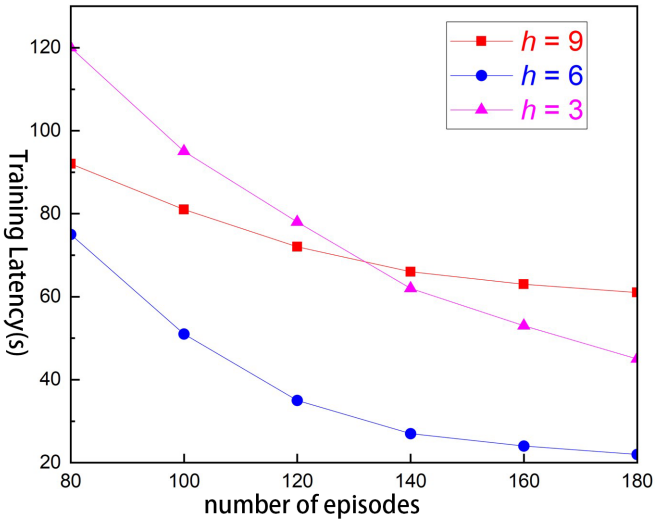

Fig. 8: Training latency versus number of episodes under different splitting layers

As shown in Fig.8, it illustrates the curves of average reward under different fixed splitting layers. Set the total computational resources of the server as $D_{total} = 150$ MFLOPS, the total storage capacity of the clients as $g = 200$ GB, and the total bandwidth as $B = 20$Hz.The splitting layer of the same model was fixed in layers 3, 6 and 9, respectively. It can be observed that when layer 6 is selected as the splitting point, the training latency exhibits faster convergence and remains lower compared to the baseline experiments. This shows that the choice of the splitting layer plays a crucial role in semantic segmentation learning. This is because selecting different layers directly impacts the complexity of both local and cloud-based models, which in turn affects the computational power and storage requirements of the clients within the semantic segmentation architecture.

As shown in Fig.9, it presents the curves of average reward under different reward rules. Set the total computational resources of the server as $D_{total} = 150$ MFLOPS, the total storage capacity of the clients as $g = 200$ GB, and the total bandwidth as $B = 20$Hz. This experiment conducts a comparative study between the PPO algorithm and the conventional

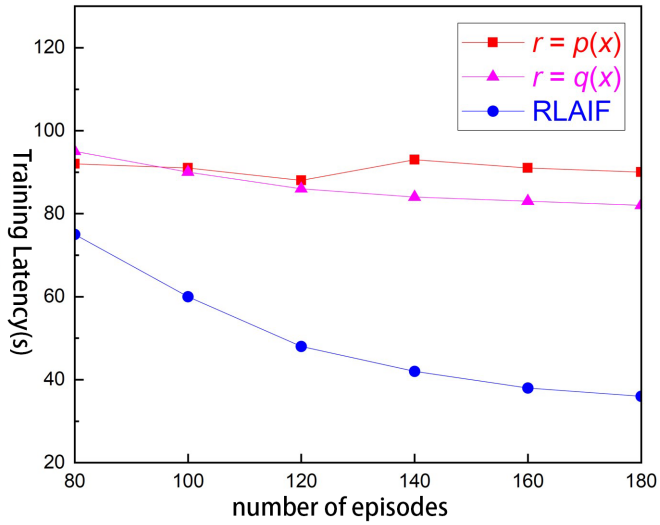

Fig. 9: Training latency versus number of episodes under different reward rules

PPO algorithm. Since this chapter formulates the semantic communication problem as a mathematical optimization task, the reward signals are generated by a function. It can be seen that traditional methods struggle to achieve low latency within the semantic segmentation learning architecture. In particular, the approach demonstrates higher training efficiency. This is because for complex optimization problems like Equation (42), designing a single effective reward signal proves challenging in reinforcement learning (RL). Such scenarios often lead to ”reward hacking,” where strategies deviate from intended objectives, and reward values fail to converge properly.

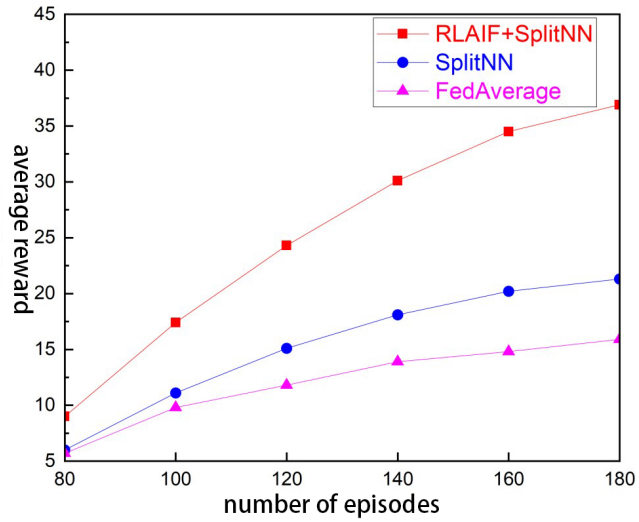

Fig. 10: Comparison of average reward versus number of episodes across different distributed learning methods

As shown in Fig.10, it presents a comparative visualization of applying our proposed method versus conventional distributed machine learning approaches to semantic model training. In particular, the SplitNN and FedAverage algorithms utilize fixed-form reward signals configured according

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

to Equation (28). It is evident that the average reward of conventional methods converges only below 20, while the method proposed in this chapter achieves significantly higher average rewards during training. This is attributed to the split learning serial training architecture, which inherently significantly reduces computational overhead on client devices, thereby enhancing reward values. In contrast, federal learning's parallel training framework necessitates frequent model adjustments based on the global federated model. Incorporating the method into semantic segmentation learning leads to higher model accuracy.

Synthesizing all experimental results demonstrates that the method proposed in this chapter effectively addresses resource allocation challenges during split learning of unimodal encoder-decoder models in semantic communication. Compared to conventional approaches, our solution reduces training latency, accelerates algorithmic convergence, and achieves higher training accuracy.

V. Conclusion

In this paper, we address the resource allocation problem in split learning of encoding-decoding models within semantic communication systems. It begins by introducing the split learning workflow and defining the communication model, including latency models, computational power models, and storage models. Subsequently, it formulates the optimization objectives along with their constraints for this study. We then define a Markov Decision Process (MDP) framework and propose a PPO algorithm enhanced, which includes an overview of the algorithmic architecture, detailed PPO procedures, and a reward mechanism that leverages large language models. Finally, simulation experiments are conducted to validate the feasibility of the proposed approach.

References

[1] Y. Che, H. Xiong, S. Han, and X. Xu, "Cache-enabled knowledge base construction strategy in semantic communications," in *2022 IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 1507–1512.

[2] L. Wang, W. Wu, F. Zhou, F. Tian, Q. Wu, and W. Saad, "A unified hierarchical semantic knowledge base for multi-task semantic communication," in *ICC 2024 - IEEE International Conference on Communications*, 2024, pp. 2937–2943.

[3] X. Wei, S. Wang, A. Zhou, J. Xu, S. Su, S. Kumar, and F. Yang, "Mvr: An architecture for computation offloading in mobile edge computing," in *2017 IEEE International Conference on Edge Computing (EDGE)*, 2017, pp. 232–235.

[4] W. Yu and J. Zhao, "Semantic communications, semantic edge computing, and semantic caching with applications to the metaverse and 6g mobile networks," in *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 983–984.

[5] R. Xie, C. Li, X. Zhou, and Z. Dong, "Asynchronous federated learning for real-time multiple licence plate recognition through semantic communication," in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[6] H. Xing, H. Zhang, X. Wang, L. Xu, Z. Xiao, B. Zhao, S. Luo, L. Feng, and Y. Dai, "A multi-user deep semantic communication system based on federated learning with dynamic model aggregation," in *2023 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2023, pp. 1612–1616.

[7] Z. Wang, L. Boccardo, and Y. Deng, "An edge-enabled wireless split learning testbed: Design and implementation," *IEEE Communications Letters*, vol. 28, no. 6, pp. 1337–1341, 2024.

[8] E. Eldeeb, M. Shehab, H. Alves, and M.-S. Alouini, "Semantic meta-split learning: A tinyml scheme for few-shot wireless image classification," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 491–501, 2025.

[9] Y. Hui, X. Chen, and L. Chen, "A novel decentralized federated split learning framework in 6g edge networks," in *2024 10th International Conference on Computer and Communications (ICCC)*, 2024, pp. 2123–2127.

[10] W. Ni, H. Ao, H. Tian, Y. C. Eldar, and D. Niyato, "Fedsl: Federated split learning for collaborative healthcare analytics on resource-constrained wearable iomt devices," *IEEE Internet of Things Journal*, vol. 11, no. 10, pp. 18 934–18 935, 2024.

[11] Z. Feng, M. Chen, X. He, N. Gao, Y. Sun, and S. Yu, "Topology-aware interpretable image semantic communication," in *2025 IEEE/CIC International Conference on Communications in China (ICCC)*, 2025, pp. 1–6.

[12] M. Chen, Y. Gu, X. He, S. Wang, F. Huang, and L. Wang, "Explainable artificial intelligence enhance image semantic communication system in 6g-iot," *IEEE Internet of Things Journal*, pp. 1–1, 2025.

[13] R. Huang, C. Qiu, M. Chen, C. Zhang, and H. Zhu, "Federated-learning-enabled cross-modal semantic communication for 6g," *IEEE Internet of Things Journal*, vol. 12, no. 20, pp. 41 608–41 624, 2025.

[14] X. Xu, K. Zheng, H. Dou, M. Chen, and L. Wang, "Federated knowledge distillation enabled image semantic communication," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 3152–3157.

[15] M. Chen, M. Liu, W. Wang, H. Dou, and L. Wang, "Cross-modal semantic communications in 6g," in *2023 IEEE/CIC International Conference on Communications in China (ICCC)*, 2023, pp. 1–6.

[16] M. Chen, L. Zhao, J. Chen, X. Wei, and M. Guizani, "Modal-aware resource allocation for cross-modal collaborative communication in iiot," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 14 952–14 964, 2023.

[17] M. Chen and Y. Xie, "Tactile signal reconstruction assisted by video processing," in *2022 14th International Conference on Wireless Communications and Signal Processing (WCSP)*, 2022, pp. 347–351.

[18] M. Chen, H. Lu, X. Xu, X.-W. Tang, Q. Fan, and C. Lou, "Novel deep packet compression for industrial internet

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Consumer Electronics. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCE.2026.3660756

of things," in *2022 IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 1628–1633.

[19] X. Xu, Y. Xu, H. Dou, M. Chen, and L. Wang, "Federated kd-assisted image semantic communication in iot edge learning," *IEEE Internet of Things Journal*, vol. 11, no. 21, pp. 34 215–34 228, 2024.

[20] M. Chen, X. Wei, J. Chen, L. Wang, and L. Zhou, "Integration and provision for city public service in smart city cloud union: Architecture and analysis," *IEEE Wireless Communications*, vol. 27, no. 2, pp. 148–154, 2020.

[21] Y. Zhang, X. Ding, H. Zhang, M. Chen, and G. Zhang, "Initial access beam management framework for leo satellite networks integrated with 5g nr," *IEEE Internet of Things Journal*, pp. 1–1, 2025.

[22] X. He, Y. Jiang, X. Xu, H. Cui, Y. Liu, M. Chen, Y. Hong, and J. Zhang, "Large language model offloading using active inference in 6g symbiotic iot," *IEEE Internet of Things Journal*, pp. 1–1, 2025.

[23] S. Hua, Y. Sun, K. Ma, L. Feng, M. Chen, Z. Yang, and M. A. Imran, "Bandwidth management in semantic communications: A tradeoff between data sensing and transmission," *IEEE Transactions on Vehicular Technology*, pp. 1–13, 2025.

[24] M. Chen, Z. Sun, X. He, B. Li, L. Wang, and S. Mumtaz, "Secure mllm semantic communication enabled air-space-ground-maritime networks," *IEEE Communications Magazine*, vol. 63, no. 7, pp. 26–33, 2025.

[25] M. Chen, Z. Sun, X. He, L. Wang, and A. Al-Dulaimi, "Llm-based semantic communication: The way from task-originated to general," *IEEE Wireless Communications Letters*, vol. 14, no. 10, pp. 3029–3033, 2025.

[26] X. He, Y. Jiang, H. Cui, Y. Liu, M. Chen, M. Guizani, and S. Mumtaz, "Qoe-driven proactive caching with drl in sustainable cloud-to-edge continuum," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10 992–11 004, 2025.

[27] M. Zeng, W. Ma, K. Zheng, W. Zhuang, and M. Chen, "Two-stage reinforcement learning empowered wireless semantic video transmission," in *2025 IEEE Wireless Communications and Networking Conference (WCNC)*, 2025, pp. 1–7.

[28] J. Wang, M. Chen, S. Yu, and X. Zhang, "Review of modeling and analysis of parasitics in power electronic converters*," *Chinese Journal of Electrical Engineering*, vol. 11, no. 1, pp. 151–173, 2025.

[29] M. Chen, C. Wang, X. He, F. Zhu, L. Wang, and A. V. Vasilakos, "Embodied artificial intelligence-enabled internet of vehicles: Challenges and solutions," *IEEE Vehicular Technology Magazine*, vol. 20, no. 2, pp. 63–70, 2025.

[30] Z. Zhang, Y. Jiang, X. Wei, M. Chen, H. Dong, and S. Yu, "Generative-ai for xr content transmission in the metaverse: Potential approaches, challenges, and a generation-driven transmission framework," *IEEE Network*, pp. 1–1, 2025.

[31] X. Ling, R. Jiang, W. Cao, M. Chen, J. Wang, Z. Ding, and X. Gao, "Exploring mec server strategy in blockchain networks: Mining for mobile users or for self," *IEEE Transactions on Mobile Computing*, vol. 24, no. 7, pp. 6030–6044, 2025.

[32] H. Lu, X. Tan, M. Chen, Z. Zhang, X. Zhang, J. Chen, X. Wei, and T. Zhao, "Cross-modal haptic compression inspired by embodied ai for haptic communications," *IEEE Transactions on Multimedia*, vol. 27, pp. 4996–5008, 2025.

[33] X. Zhang and M. Chen, "Deep learning empowered real-time haptic communications for iot," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 2, pp. 5606–5618, 2025.

[34] Y. Rong, Y. Mao, X. He, and M. Chen, "Large-scale traffic flow forecast with lightweight llm in edge intelligence," *IEEE Internet of Things Magazine*, vol. 8, no. 1, pp. 12–18, 2025.

[35] B. Li, Y. Zou, T. Wu, Z. Zhang, M. Chen, and Y. Jiang, "Security and reliability tradeoff of noma based hybrid satellite-terrestrial network with a friendly jammer," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3439–3444, 2025.

[36] X. Xu, K. Zheng, H. Dou, M. Chen, and L. Wang, "Federated knowledge distillation enabled image semantic communication," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 3152–3157.

[37] X. Zhang, X. He, M. Chen, and L. Wang, "Reinforcement learning from human-like feedback enhances semantic communication with multimodal llms," in *2024 16th International Conference on Wireless Communications and Signal Processing (WCSP)*, 2024, pp. 1491–1496.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.